System Architecture & Implementation Plan: NexusProxy

High-Performance, Self-Hosted API Key Rotation & Gateway

1. The Core Problem

Many independent developers, student developers, and hobbyists building applications (such as AI automation tools, LLM agents, or data scraping systems) rely on the free tiers of premium APIs (e.g., Brave Search, OpenAI, Anthropic, or Serper).

However, these free allowances are heavily throttled by strict rate limits or tiny monthly request caps. To get around this, developers frequently create 3 to 4 separate free accounts to pool more daily or monthly requests. Managing these keys manually, however, creates massive friction:

- **Brittle Applications:** The client application crashes or hangs the exact moment a specific key hits its request cap (429 Too Many Requests).
- **Manual Overhead:** The developer has to manually stop their application, swap out environment variables or configuration files with a new key, and restart the system.
- **Codebase Pollution:** Hardcoding complex key-switching, error-handling, and fallback logic directly into a project makes the codebase messy, unmaintainable, and difficult to reuse across different projects.

2. The Solution

NexusProxy is an open-source, self-hosted developer tool that acts as a local smart reverse proxy. It consolidates multiple free-tier tokens into a single, unified virtual API key and local endpoint.

Instead of configuring your client application or script to talk directly to an external premium service, you point it to NexusProxy running locally on your machine or home server. NexusProxy handles all the heavy lifting in the background—tracking remaining request allocations, evaluating response headers, managing cooldown periods, and hot-swapping keys mid-flight without your client application ever knowing a switch happened.

- **Zero External Footprint:** Because it is entirely self-hosted, your API keys remain completely local. This eliminates third-party security risks and prevents premium API providers from blacklisting a centralized cloud platform IP.
- **Predictive Routing:** It monitors live header data to switch keys *before* a failure occurs, rather than waiting for a request to drop.

Provider Agnosticism: It abstracts different upstream platforms so you can seamlessly bounce between entirely different providers if one service goes down completely.

Extreme Efficiency: Built to run as a highly optimized, single-purpose service that consumes virtually zero idle CPU and minimal RAM, making it perfect for lightweight backgrounds or local docker stacks.

3. Core Engine Architecture

The system is broken down into four intelligent, decoupled sub-systems:

A. The Virtual Listener Engine: Maps a local port on your host machine to accept standard payload calls. It acts as the traffic controller, intercepting outgoing requests from your application, stripping out the localized authorization token, and preparing the payload to be forwarded to the real target provider.

B. The Predictive Routing State-Machine: An ultra-fast, in-memory state engine that keeps a live scorecard of every key in your pool. Instead of waiting blindly for an application error, it reads the standard response metadata headers sent back by premium APIs (such as remaining quotas and reset windows). It continuously updates its internal tracking metrics to route the very next application request to whichever key has the healthiest available balance.

C. The Intelligent Circuit Breaker: If an unpredicted rate limit drops or a key suddenly fails with a 429 error, the Circuit Breaker traps the failure instantly. It prevents the error from leaking back to your client app, extracts the required wait time from the API's headers, flags that specific key as 'Cooling Down', and immediately duplicates and reroutes the original request payload to an alternative, healthy standby key in the pool.

D. The Translation & Normalization Layer: An advanced abstraction system that allows you to standardize your application logic. If you are using search tools, for example, you can query a single universal payload format. If your Brave Search keys run completely dry, the translation layer can instantly re-map your request format to fit a Google Serper or Bing API structure on the fly, transforming the output back into your expected unified response format.

4. System Configuration Structure

The tool utilizes a highly structured, human-readable configuration file that loads into memory at system boot. It maps out:

Gateway Server Settings: Defines local hosting ports, debugging modes, and internal performance tracking flags.

Routing Policies: Allows the developer to set selection behaviors, such as strict Round-Robin rotation, Priority-ordered mapping, or Predictive Quota routing.

Provider & Key Management: A clean matrix where developers can categorize keys under specific service types, assign individual key priorities, and drop in as many free tier tokens as they want.

Contract Mapping Definitions: Outlines how incoming universal request parameters should be structurally translated into specific provider schemas when multi-provider fallback routing is active.

5. Distribution & Deployment Strategy

To guarantee that the proxy can be dropped into any developer environment regardless of whether they are building a web app, a mobile app, or data pipelines, it is packaged as a Single Multi-Stage OCI/Docker Container.

- **Minimal Footprint:** The final image utilizes an optimized, empty runtime layer containing absolutely no background environment baggage. The total container size sits at an incredibly lean ~15 Megabytes.
- **Environment Agnostic:** It runs exactly the same on a local development laptop, an on-premise home server environment, or an isolated cloud instance.
- **Single Command Spin-up:** Developers can mount their local configuration file and expose the port with a single terminal command, making setup instantaneous.

6. Milestone Execution Roadmap

Phase 1: Core Engine & In-Memory Routing (Milestone 1)

- Establish the core network routing pipeline and incoming HTTP listener.
- Create the configuration parser to smoothly ingest and watch local YAML settings updates.
- Implement the thread-safe in-memory map to manage basic request forwarding and token injection.

Phase 2: Predictive Balancing & Error Resiliency (Milestone 2)

- Write the parsing filters to extract rate-limiting information from standard upstream API response headers.
- Implement the Circuit Breaker logic to catch 429 errors and automatically switch keys mid-flight.
- Introduce the automated cooldown timers to safely reinstate keys into the active pool once their rate limit windows reset.

Phase 3: Translation Layers & Monitoring Dashboard (Milestone 3)

- Build request/response transformation mapping for primary use cases (e.g., standard web search and LLM completion models).
- Embed a lightweight, static HTML status interface to visually track live request counts, active key statuses, and estimated costs saved.
- Package the finalized engine into the cross-platform optimized Docker distribution setup.